

Áp dụng Nguyên lý và Mẫu thiết kế cho UC "Xử lý yêu cầu nhập hàng"

I. Áp dụng Mẫu thiết kế (Design Patterns)

1. Mẫu thiết kế Facade (Mặt tiền)

Thiết kế ban đầu:

Giao diện JavaFX của bộ phận đặt hàng quốc tế thao tác trực tiếp với nhiều lớp nghiệp vụ khác nhau như:

- `SaleRequestController`
- `OverseasOrderController`
- `OverseasOrderSuggestionEngine`
- các repository như `SiteRepository`, `SiteMerchandiseRepository`

Điều này làm cho lớp giao diện như `OverseasDashboard` phải biết quá nhiều chi tiết về cách xử lý yêu cầu nhập hàng, từ việc gợi ý site, lập kế hoạch đơn hàng, đến tạo đơn thật. Nếu để giao diện gọi trực tiếp toàn bộ logic này, lớp UI sẽ bị phình to và khó bảo trì.

Cách chỉnh sửa áp dụng Pattern:

Trong code hiện tại, logic nghiệp vụ đã được gom phần lớn vào một lớp điều phối trung tâm:

- `OverseasOrderController`
- `OverseasOrderService`

Trong đó:

- `OverseasDashboard` chỉ gọi các hàm như:
 - `suggestionEngine.suggest(request)`
 - `overseasOrderController.createManualSiteOrder(...)`
 - `overseasOrderController.getSiteOrdersBySaleRequest(...)`
- `OverseasOrderController` đóng vai trò lớp trung gian che giấu chi tiết xử lý phía dưới.
- `OverseasOrderService` giữ toàn bộ logic nghiệp vụ chính:
 - `processSaleRequest(...)`
 - `createManualSiteOrder(...)`
 - `getSiteOrdersBySaleRequest(...)`

Ý nghĩa đem lại:

- Giảm sự phụ thuộc giữa giao diện và tầng nghiệp vụ.
- Che giấu chi tiết xử lý bên dưới, giúp luồng nghiệp vụ rõ ràng hơn.
- Nếu sau này cần đổi cách tạo đơn, thêm bước kiểm tra, hoặc bổ sung thông báo, ta chủ yếu sửa trong `OverseasOrderService` hoặc `OverseasOrderController`, không cần đụng nhiều vào `OverseasDashboard`.

2. Mẫu thiết kế DTO (Data Transfer Object)

Thiết kế ban đầu:

Nếu giao diện làm việc trực tiếp với các entity như:

- `SaleRequest`
- `SaleRequestItem`
- `SiteOrder`
- `SiteOrderItem`

thì dữ liệu tầng database sẽ bị lộ lên UI, đồng thời giao diện phải phụ thuộc vào cấu trúc DB thật.

Cách chỉnh sửa áp dụng Pattern:

Project đã dùng các DTO để truyền dữ liệu giữa các tầng:

- `SaleRequestDTO`
- `SaleRequestItemDTO`
- `SiteOrderDTO`
- `SiteOrderItemDTO`
- `ProcessResult`

Trong code thật:

- `SaleRequestService.buildDTO(...)` chuyển `SaleRequest` và `SaleRequestItem` sang `SaleRequestDTO`
- `OverseasOrderService.buildSiteOrderDTO(...)` chuyển `SiteOrder` và `SiteOrderItem` sang `SiteOrderDTO`

Ý nghĩa đem lại:

- Tách biệt rõ entity và dữ liệu hiển thị.

- Không để giao diện phụ thuộc trực tiếp vào bảng dữ liệu.
- Chỉ truyền những thông tin cần thiết cho màn hình, tránh dư thừa dữ liệu.
- Dễ mở rộng khi muốn thay đổi giao diện hoặc API sau này.

3. Mẫu thiết kế Strategy-like cho gợi ý site

Thiết kế trong code:

Phần gợi ý site trong `OverseasOrderSuggestionEngine` thể hiện một thuật toán chọn site riêng biệt, tách khỏi giao diện và controller.

Các hàm chính:

- `suggest(SaleRequestDTO request)`
- `buildItemPlan(SaleRequestItemDTO item)`
- `bestOptionForItem(...)`

Ý nghĩa:

- Thuật toán gợi ý site đã được tách riêng thành một thành phần độc lập.
- Nếu sau này đổi cách chọn site, ví dụ ưu tiên theo chi phí thay vì thời gian, ta chỉ cần sửa trong `OverseasOrderSuggestionEngine`.
- Đây là biểu hiện rất gần với tư tưởng của Strategy: đóng gói thuật toán vào một lớp riêng để dễ thay thế.

II. Áp dụng Nguyên lý thiết kế (SOLID Principles)

1. Nguyên lý Đơn trách nhiệm (SRP - Single Responsibility Principle)

Cách áp dụng trong use case này:

Mỗi lớp trong kiến trúc của bạn đang đảm nhiệm một nhóm trách nhiệm tương đối rõ ràng:

- `OverseasDashboard`
Chỉ lo giao diện, nhận sự kiện người dùng, hiển thị danh sách, popup, và điều hướng màn hình.
- `OverseasOrderSuggestionEngine`
Chỉ lo tính toán và gợi ý site phù hợp cho từng mặt hàng.
- `OverseasOrderController`
Chỉ làm lớp điều phối giữa giao diện và nghiệp vụ, đồng thời kiểm tra quyền.

- `OverseasOrderService`

Chỉ chứa logic xử lý yêu cầu nhập hàng:

- chạy thuật toán phân bổ site
- tạo đơn
- cập nhật trạng thái
- xử lý hủy

- `SaleRequestRepository`, `SiteOrderRepository`, `SiteMerchandiseRepository`, ...

Chỉ lo truy xuất dữ liệu SQL.

Ý nghĩa đem lại:

- Code dễ đọc, dễ sửa.
- Mỗi lớp có một lý do thay đổi chính.
- Khi cần đổi thuật toán gợi ý site, chỉ sửa `OverseasOrderSuggestionEngine`.
- Khi cần đổi cách lưu dữ liệu, chỉ sửa repository.

2. Nguyên lý Mở - Đóng (OCP - Open/Closed Principle)

Cách áp dụng:

Use case xử lý yêu cầu nhập hàng đã được thiết kế sao cho luồng nghiệp vụ có thể mở rộng mà không cần viết lại toàn bộ cấu trúc hiện có.

Ví dụ:

- Nếu muốn bổ sung thêm cách tạo đơn mới:
 - tự động theo gợi ý
 - tạo thủ công
 - tạo theo mức ưu tiên site

thì có thể mở rộng trong `OverseasOrderService` hoặc bổ sung phương thức mới trong `OverseasOrderController`, trong khi UI vẫn giữ cấu trúc chính.

Ý nghĩa đem lại:

- Hệ thống dễ mở rộng.
- Tránh sửa quá nhiều vào code cũ.
- Giảm nguy cơ gây lỗi dây chuyền.

3. Nguyên lý Đảo ngược phụ thuộc (DIP - Dependency Inversion Principle)

Cách áp dụng trong use case này:

Các lớp cấp cao như:

- `OverseasDashboard`
- `OverseasOrderController`
- `OverseasOrderService`

không làm việc trực tiếp với SQL hay database vật lý, mà thông qua các lớp repository trung gian:

- `SaleRequestRepository`
- `SaleRequestItemRepository`
- `SiteRepository`
- `SiteMerchandiseRepository`
- `SiteOrderRepository`
- `SiteOrderItemRepository`

Tương tự, `OverseasDashboard` không xử lý chi tiết thuật toán gợi ý site mà gọi:

- `OverseasOrderSuggestionEngine`

Ý nghĩa đem lại:

- Giảm phụ thuộc trực tiếp giữa tầng cao và tầng thấp.
- Dễ kiểm thử unit test hơn.
- Có thể thay đổi cách truy xuất dữ liệu mà không ảnh hưởng mạnh đến giao diện hay controller.

4. Nguyên lý Phân tách giao diện lớn (ISP - Interface Segregation Principle)

Cách nhìn trong project:

Dù project hiện tại chưa dùng interface nhiều, nhưng tinh thần của ISP thể hiện ở chỗ các lớp được tách theo chức năng nhỏ:

- `SaleRequestController` lo các chức năng liên quan yêu cầu nhập hàng từ bộ phận sales
- `OverseasOrderController` lo xử lý yêu cầu nhập hàng cho bộ phận đặt hàng quốc tế
- `SaleRequestService` và `OverseasOrderService` tách biệt trách nhiệm

Ý nghĩa đem lại:

- Không có một lớp “siêu to” phải gánh quá nhiều API.
- Mỗi module chỉ cung cấp đúng những hàm cần thiết cho nhóm chức năng của mình.

III. Kết luận

Với use case “Xử lý yêu cầu nhập hàng”, project đã thể hiện khá rõ:

- Facade qua lớp điều phối `OverseasOrderController` và luồng gọi từ `OverseasDashboard`
- DTO qua `SaleRequestDTO`, `SiteOrderDTO`, `SaleRequestItemDTO`, `SiteOrderItemDTO`
- SRP qua việc tách UI, controller, service, repository và engine gợi ý
- DIP qua việc tầng trên không làm việc trực tiếp với SQL mà thông qua repository
- OCP nhờ cấu trúc đủ linh hoạt để mở rộng thêm kiểu xử lý đơn trong tương lai